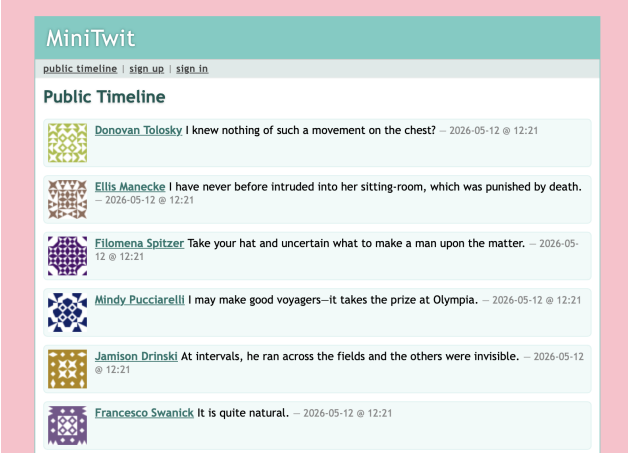
DevOps, Software Evolution and Software Maintenance, MSc



Group K - Pat Myaz

Carmen Alberte Nielsen | alcn@itu.dk
Casper Storm Frøding | csfr@itu.dk
Mads Christian Nørklit Jensen | macj@itu.dk
Mathilde Julie Gonzalez-Knudsen | mgon@itu.dk
Max Brix Koch | mbko@itu.dk

18-05-2026

Contents

1 System's Perspective **3**

1.1 Design and Architecture . . . 3

1.1.1 Sequence diagram of our system . . . 3

1.2 Dependencies . . . 3

1.3 Current State . . . 8

2 Process' Perspective **8**

2.1 CI/CD . . . 8

2.2 Monitoring . . . 10

2.2.1 Dashboards . . . 11

2.3 Logging . . . 13

2.3.1 What our current logging implementation is missing . . . 14

2.4 Security . . . 14

2.5 Availability . . . 15

3 Reflection Perspective **15**

3.1 Evolution and Refactoring . . . 15

3.2 Operation . . . 15

3.3 Maintenance . . . 15

4 Use of AI **15**

5 Appendix **16**

5.1 Appendix for CI/CD workflows . . . 16

1 System's Perspective

A description and illustration of the:

- Design and architecture of your ITU-MiniTwit systems.
- All dependencies of your ITU-MiniTwit systems on all levels of abstraction and development stages. That is, list and briefly describe all technologies and tools you applied and depend on.
- Describe the current state of your systems, for example using results of static analysis and quality assessments.

1.1 Design and Architecture

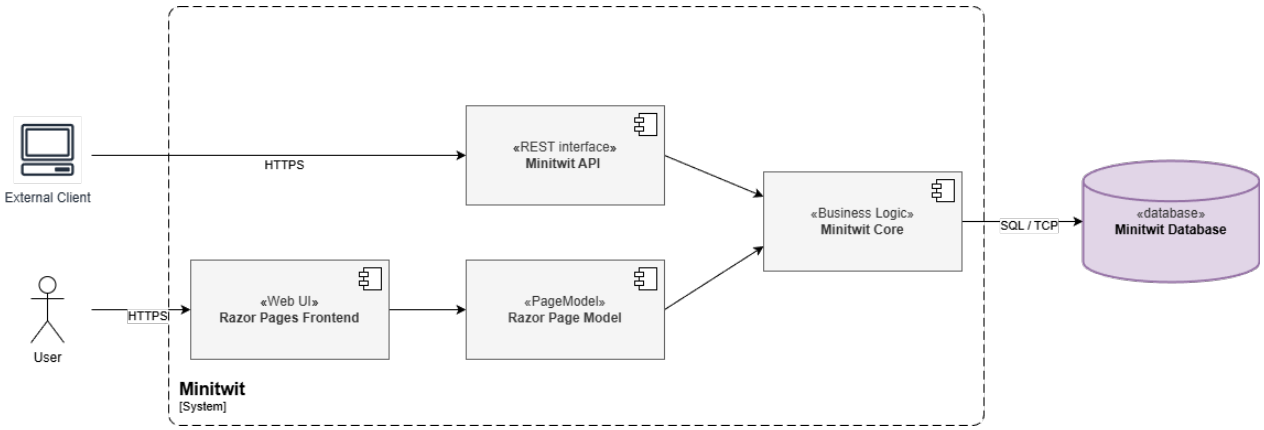


Figure 1: Component and Connectors View

1.1.1 Sequence diagram of our system

In Figure 3, we show how our system handles a post message request from a user, via the rendered Razor page.

In Figure 4, we show how our system handles a post message request, via the API.

1.2 Dependencies

Technologies:

Dependency type	Technology	Purpose
Infrastructure	DigitalOcean	Cloud Hosting Droplets and Firewall
OS	Ubuntu (ubuntu-22-04-x64)	Operating system on DigitalOcean Droplets
Orchestration	Docker Swarm	Container Clustering
Containerization	Docker	Container Runtime Environment
Containerization	Docker Compose	Multi-container configuration and local orchestration
Container Registry	DockerHub	Storing repositories of our minitwit docker images

Dependency type	Technology	Purpose
Container Hardened Images	dhi.io	Docker hardened images for .NET 10
Container Base Image	Python3.12/alpine	Lightweight runtime for Python containers
Container Base Image	mysql:8.0	MySQL base image for docker container
Container Base Image	grafana/grafana:12.1	Grafana base image for docker containers
Container Base Image	prom/prometheus:v3.5.1	Prometheus base image for docker containers
Container Base Image	docker.elastic.co/elasticsearch/elasticsearch:7.15.0	ElasticSearch base image for docker containers
Container Base Image	curlimages/curl	Curl base image for docker containers used for healthchecks
Provisioning Tool	Vagrant	Infrastructure-as-Code for VM provisioning
Provisioning Tool	DropletKit	Integration between Vagrant and DigitalOcean API
Language	ruby	Language used for Vagrant configuration
Domain Management	name.com	Domain name registration and managing
Reverse Proxy	Traefik	Load balancing and reverse proxy for routing traffic
Security	Let's Encrypt	Automated SSL/TLS certificate provisioning
Security	Fail2Ban	Protects droplets by banning malicious IPs
Scripting	bash	Shell scripts for automated tasks
Monitoring	Prometheus (Server)	Metrics collection
Monitoring	Grafana	Visualization and dashboarding of metrics and logging
Logging	ElasticSearch	Log storage, indexing and search
Runtime Framework	.NET 10	Application runtime and framework
Language	C#	Code language for Minitwit
Database	MySQL 8.0	Relation database for application data
Styling Language	CSS	Frontend styling language
Frontend Framework	Bootstrap	CSS framework for UI design
Scripting Language	JavaScript	Client-side logic and interaction
Version Control	Github	Version control platform
CI/CD	Github Actions	CI/CD workflows
Security	CodeQL	Static security analysis of code
Security	Trivy	Vulnerability scanning for containers and Dockerfiles
Analysis Tool	Dockle	Analysis of Dockerfiles
Analysis Tool	Codacy	Automated code quality analysis
Analysis Tool	SonarQube	Code quality and static analysis
Runtime Dependency	Java (JDK17)	Required runtime for SonarQube
Documentation Tool	TinyTex	Lightweight LaTeX distribution for report generation

Dependency type	Technology	Purpose
Documentation Tool	Pandoc	Document conversion and report generation
Language	Python 3	Code language for UI and API tests
Web browser	Firefox	Web browser for automated UI tests
Testing Tool	Geckodriver	WebDriver implementation for controlling Firefox in tests

.NET depencies:

Dependency type	Package	Purpose
Web Framework	ASP.NET Core	Razor Pages
API Tooling	OpenAPITools	Generating and documenting APIs
Code Analysis	.NET Analyzers	Analysing .NET code
Code Analysis	Roslynator	Linting .NET code
ORM	EntityFramework Core	Database abstraction and object-relational mapping
ORM Extension	Pomelo.EntityFrameworkCore.MySql	MySQL provider for EntityFramework Core
Monitoring	Prometheus (Client library)	Exposing application metrics
Logging Sink	Elastic.Serilog.Sinks	Sending Serilog's to ElasticSearch
Logging Library	Serilog	Structured logging and log management
Testing Framework	Xunit	Unit testing framework
Testing	Microsoft.NET.Test.Sdk	Enables running and executing tests
Testing	Microsoft.AspNetCore.Mvc.Testing	Supports integration testing of ASP.NET Core apps
Test Runner	xunit.runner.visualstudio	Runs xUnit tests in development environments

Python dependencies:

Dependency type	Package	Purpose
Testing Framework	pytest	Running and managing automated tests
Browser Automation	selenium	Automating browser interactions for testing
Web Framework	Flask	Making HTTP requests to APIs and services
HTTP Client	requests	Lightweight web framework for building APIs/services
Database Driver	PyMySQL	Connecting to and interacting with MySQL databases

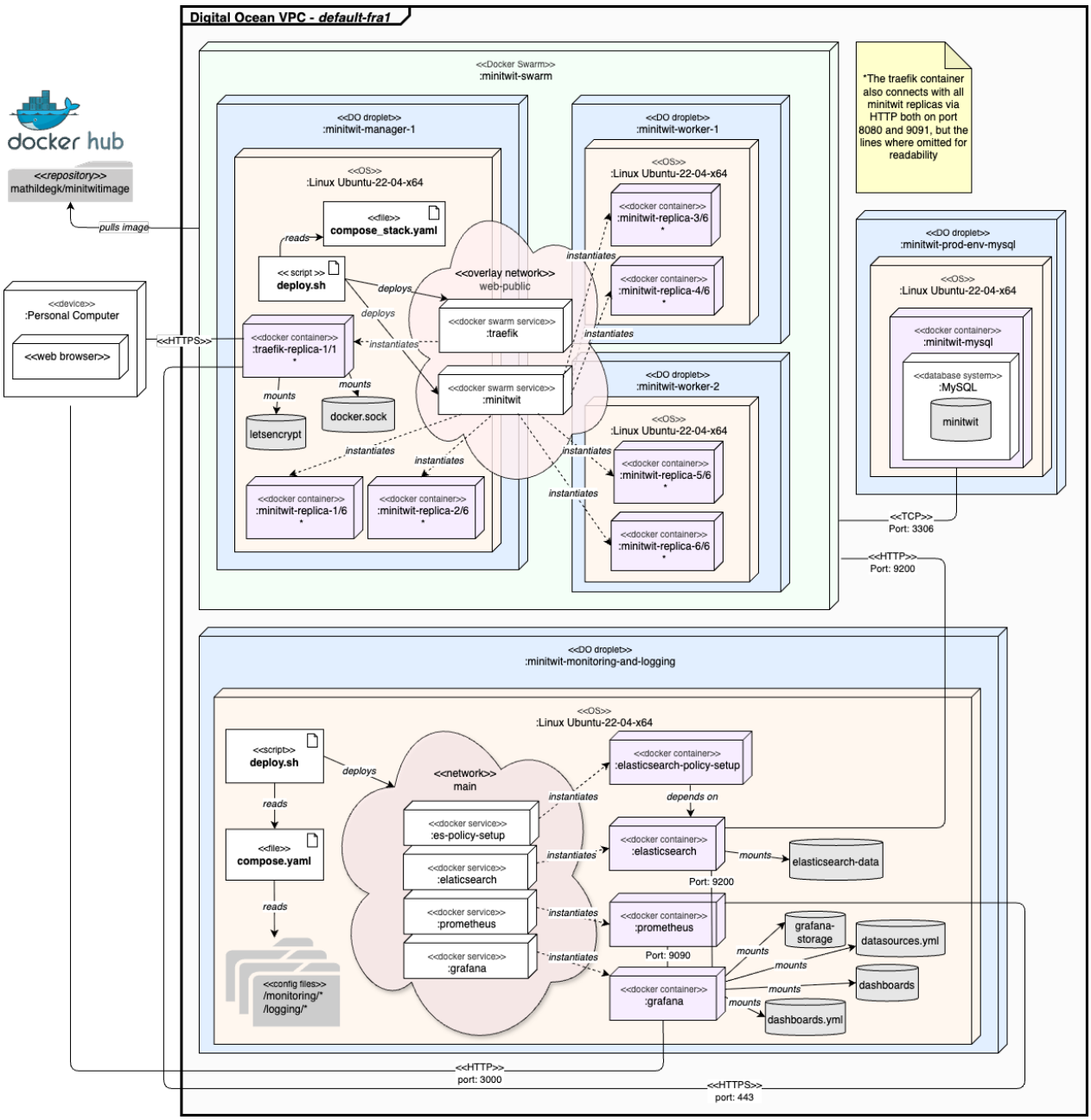


Figure 2: Allocation View

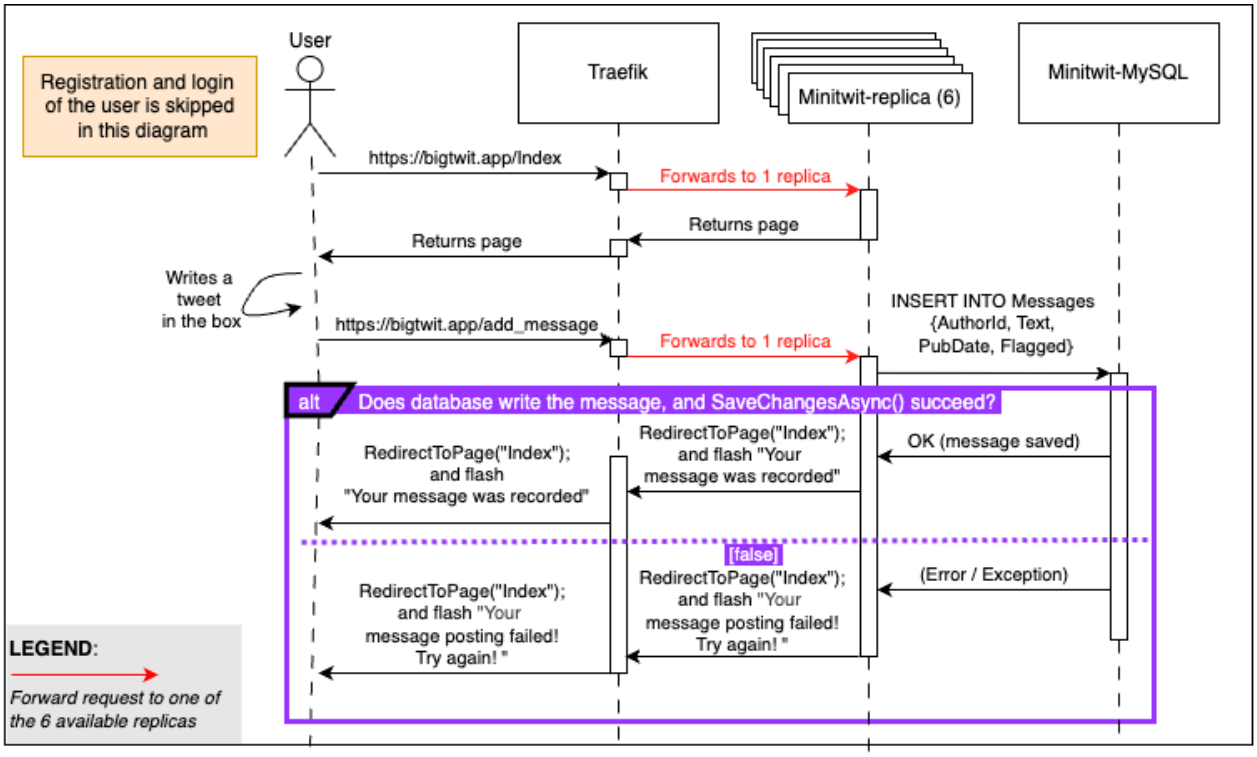


Figure 3: Sequence Diagram of a user posting a message

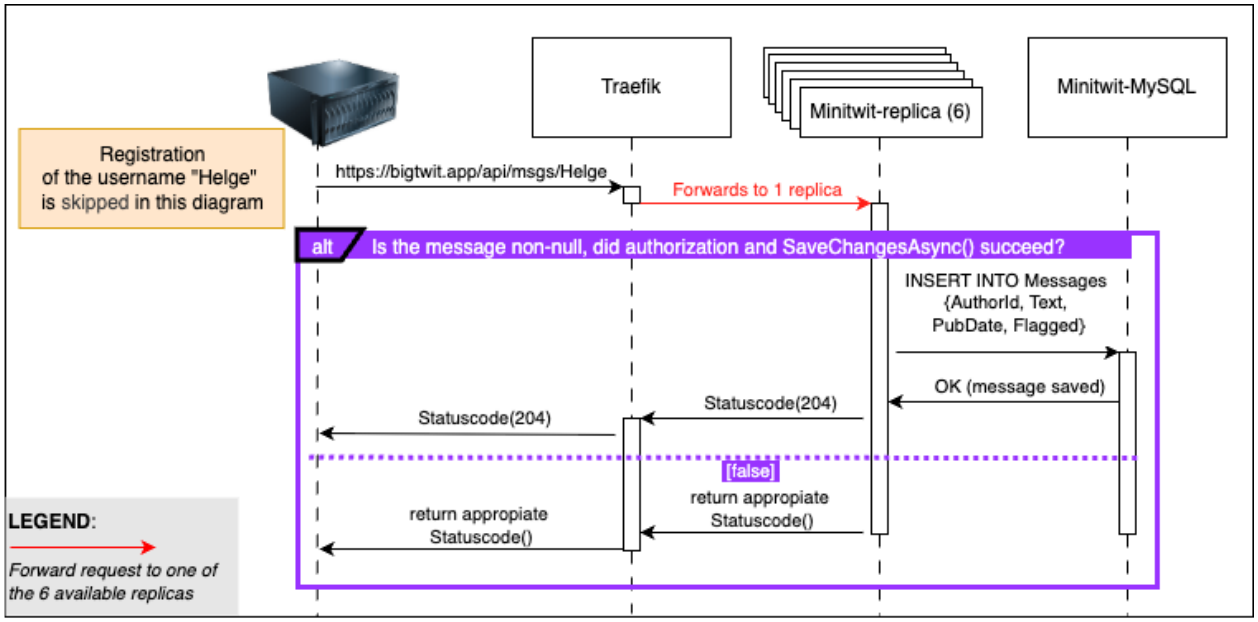


Figure 4: Sequence Diagram of an API posting a message

1.3 Current State

2 Process' Perspective

This perspective should clarify how code or other artifacts come from idea into the running system and everything that happens on the way.

In particular, the following descriptions should be included:

- A complete description and illustration of stages and tools included in the CI/CD pipelines, including deployment and release of your systems.
- How do you monitor your systems and what precisely do you monitor?
- What do you log in your systems and how do you aggregate logs?
- Brief description of how you security hardened your systems.
- How do you handle availability and scaling in your systems?

2.1 CI/CD

For our CI/CD pipelines we decided to use GitHub Actions, since our codebase is hosted on GitHub and it then seemed like the simplest option to get started, compared to using external pipeline services. We have created several workflows that run based on different event triggers. As mentioned in section 1.1, we have both a production environment and an identical test environment; both of them are used in our CI/CD pipelines.

Our CI pipeline is triggered on a pull request to main and includes both code analysis and automated testing. If the tests pass, a new image of the MiniTwit application is pushed to DockerHub and deployed to the test environment. This allows us to manually verify that the new changes from the PR integrate well into our existing application, and lowers the likelihood of unexpected issues when these changes are deployed to production.

Our CD pipeline is triggered by a push to main. This pipeline includes the same steps as our CI pipeline, except that the final step now deploys the MiniTwit image to production environment. This means our system is continuously deployed to production whenever changes have been reviewed and pass several checks.

In Figure 5, a simplified diagram of the CI/CD pipeline is shown. Only the names of the stages are shown, the steps executed in each workflow are not included here.

In Figure 6 is an illustration of the automated testing workflow. This workflow is reused in both the workflow for deployment to test environment and the one for deployment to production environment.

Figure 7 illustrates the workflow that deploys to production environment. As mentioned before, it uses the workflow for tests shown in figure 6.

The workflow for deploying to test environment is very similar to Figure 7, an illustration is included in appendix 5.1. Illustrations of the workflows for CodeQL, SonarQube and Trivy can be found in the same place.

We also have a workflow for creating a draft for a new release. It is triggered whenever a new versioning tag is pushed. We would have liked to improve this workflow by finding some way to automate more of it, since it right now requires manual interference both for creating and pushing a tag, and then reviewing and publishing the release. This workflow is illustrated in appendix 5.1.

We have also set up a workflow with Pandoc which generates a new PDF for the report on every PR and push to main where changes have been made in the /report folder. This is also illustrated in appendix 5.1.

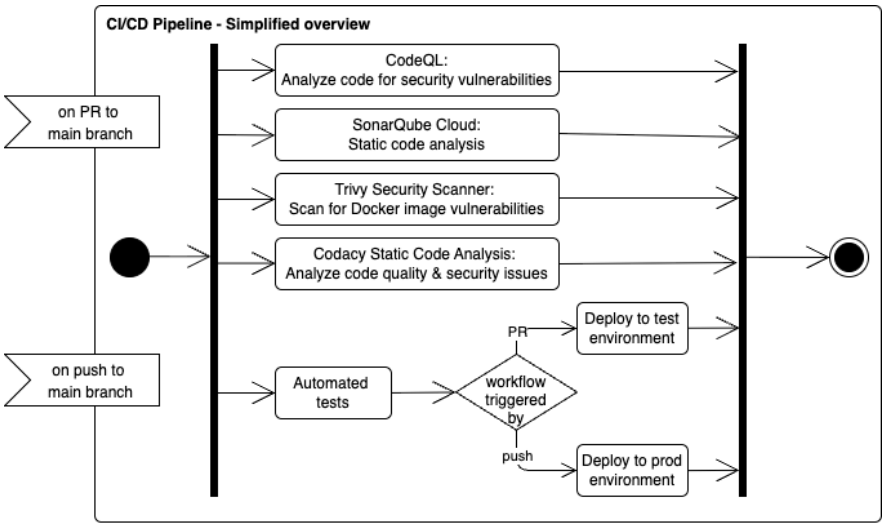


Figure 5: Simplified CI/CD pipeline with no individual workflow steps shown

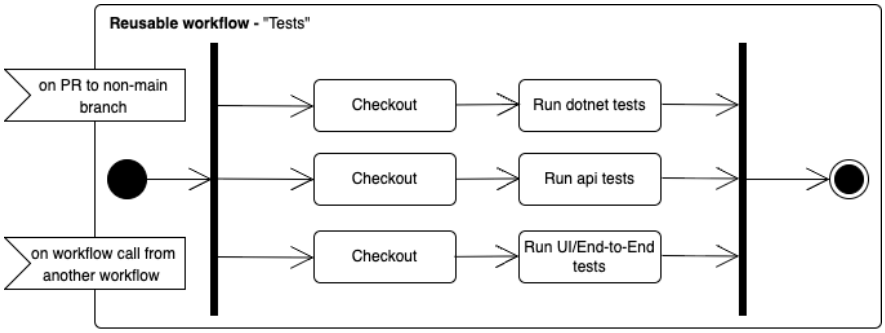


Figure 6: Workflow running automated tests

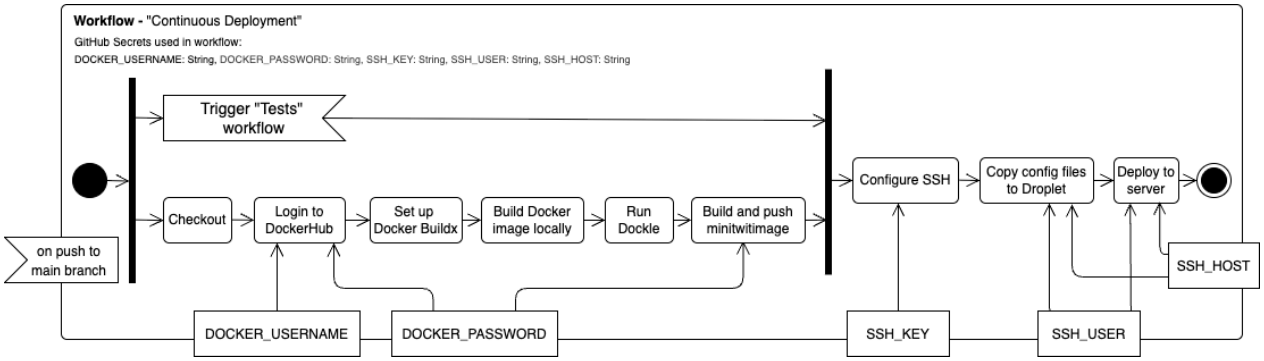


Figure 7: Workflow testing and deploying to production

2.2 Monitoring

Figure 8 is a sequence diagram showing the process of collecting, forwarding, and visualizing application metrics in the monitoring system.

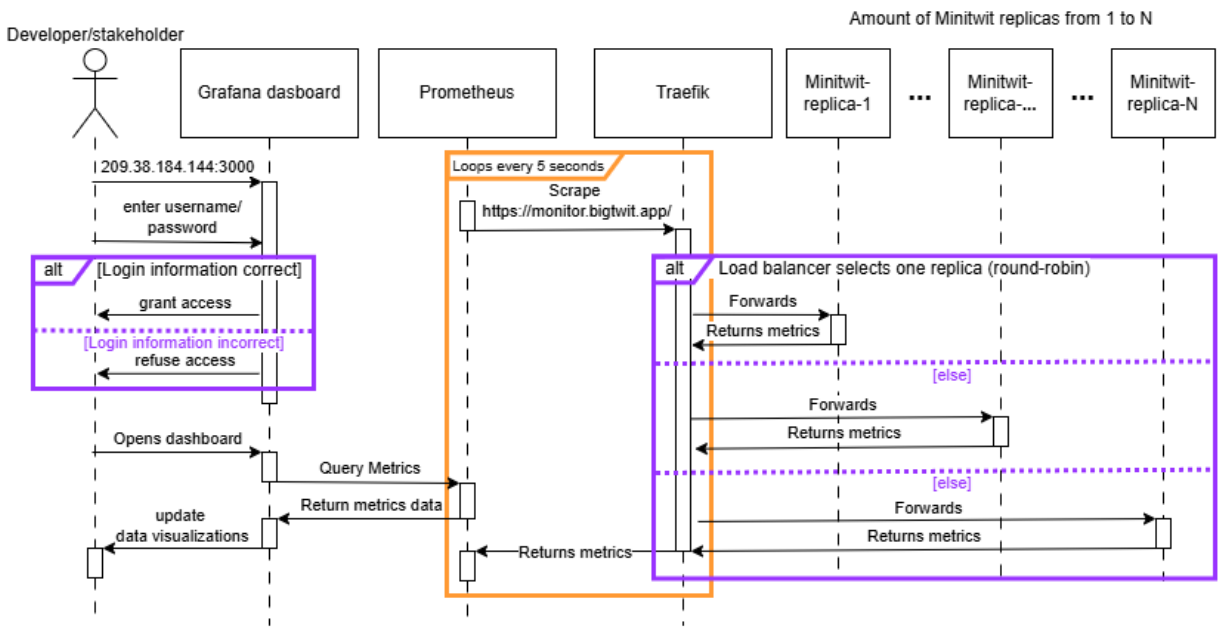


Figure 8: A sequence diagram of how monitoring works for Minitwit

To ensure we do not leak information on our metrics of Minitwit, we have set a username and password on Grafana, that is automatically set on creation. In addition to this, only the Monitor's droplet IP adress is whitelisted to scrape https://monitor.bigtwit.app/, done via Traefik, so no other attacker would be able to access this information.

On this metrics endpoint we use `app.UseHttpMetrics();`, which writes things to the endpoint such as total amount of HTTP requests received, the duration of these requests, and more HTTP specific metrics. In addition to this, we added 2 "gauges" to the metrics, that showed the amount of tweets and users the app has. This data is cached, so if another metrics request is received in less than 30 seconds, we re-use the old data for tweets, and followers. This is done to not overwork the database, with requests to count the total amount of tweets and users.

```
// Before collecting metrics, we want to update the total amount of tweets and users,
// so we get the latest values when scraping
DateTime _lastUpdate = DateTime.MinValue;
Metrics.DefaultRegistry.AddBeforeCollectCallback(async (cancel) =>
{
  if ((DateTime.Now - _lastUpdate).TotalSeconds < 30) return; // Skip if too recent
  //The DbContext is a scoped service, so we need to create a scope to get it
  using (var scope = app.Services.CreateScope())
  {
    var context = scope.ServiceProvider.GetRequiredService<MinitwitContext>();

    // Query the database for the absolute latest numbers
    var totalTweets = await context.Messages.CountAsync(cancel);
    var totalUsers = await context.Users.CountAsync(cancel);

    // Update the Gauges
    MinitwitMetrics.TotalTweets.Set(totalTweets);
    MinitwitMetrics.TotalUsers.Set(totalUsers);
    _lastUpdate = DateTime.Now;
  }
});
```

Figure 9: The source code for caching, in the metrics

As can be seen in Figure 9 and Figure 8, we have a hybrid of push/pull monitoring, as Minitwit pushes some internal data to a metrics endpoint, and Prometheus pulls from this endpoint.

As can be seen in the Figure 8, Traefik load balances between each container, which creates this fluctuating graph, that can be seen in Figure 10 and Figure 11 and Figure 12 and Figure 15 under here, due to Prometheus not knowing exactly what container it has scraped, and can therefore not make an average.

To mitigate this error, we should have moved our Prometheus container from its' own droplet to the Manager droplet, and then from here it would be able to scrape all the individual replicas, and find averages over these, via the Docker network, and communicate this to Grafana on the monitoring droplet. However, this would still be a problem due to the workers being on different droplets than the manager, therefore we would need a package that could discover the exact location of each replica, for example the (prometheus-docker-sd package).

In addition to this we also wanted to monitor all of our services, i.e. not "just" the webservice, but also the database, to make it easier to determine where an error/bottleneck is located, however we did not get to implement this.

Due to us developers not wanting to constantly monitor our system, we have set up an *Uptime Check*, that monitors our droplets, and gives everyone in the team a notification if:

- https://bigtwit.app is down
- there is latency over 1000ms when accessing https://bigtwit.app
- The SSL certificate of bigtwit.app is about to expire.(However Traefik should automatically renew this certificate, before it expires.)

In addition to this we have added a ressource alert, that monitors our test database and production database, to see when it crosses over 70% CPU in over 5 minutes, which could indicate an attack, or a rapid increase in requests that it maybe cannot handle.

2.2.1 Dashboards

To see the dashboard shown in Figures 10,11,12,13,14,15, go to 46.101.69.11:3000, where the dashboard and logs can be viewed. The credentials are specified in the post made by Helge.

We made 2 different dashboards, one made with a developer focus (Infrastructure Health), and one made with a Business stakeholder focus(Product Performance).

2.2.1.1 Minitwit Infrastructure Health Dashboard

In Figure 10,11,12, are the images of the dashboard and descriptions of the information it provides for the developers:

- Immediate Root Cause Analysis: If the Status is UP but the Error Rate is spiking, developers know the app is running but the code or database is failing.
- Capacity Planning: Requests per second and Total Requests help developers understand if the current infrastructure (CPU/RAM) can handle the load or if they need to scale.
- Performance Regressions: By monitoring Latency(P95) and HTTP Success rate, developers can see if a recent code deployment made the app slower, even if it didn't *break* it.

We have, in the source code, created a */healthz* endpoint, that shows when Minitwit is up and running, however this is only used for spinning up the system locally with docker compose. Our dashboard's health check is currently based on whether or not Prometheus was able to scrape the target, this should instead be based on the */healthz* endpoint to seperate it from Prometheus.

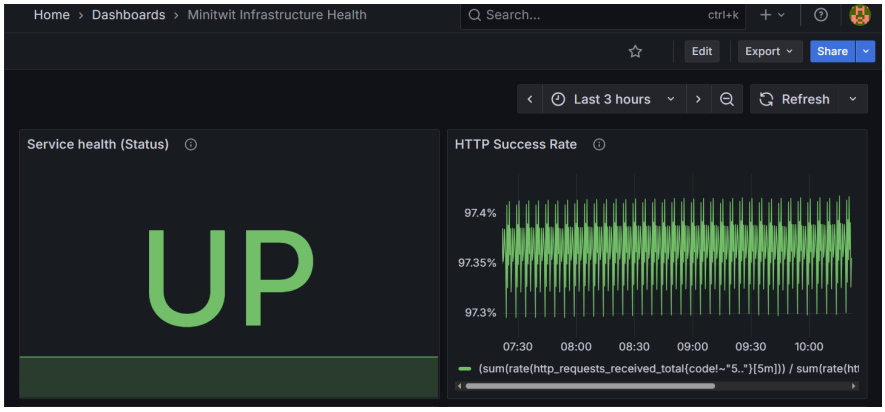


Figure 10: Minitwit Infrastructure Health dashboard - Service health and HTTP Success Rate

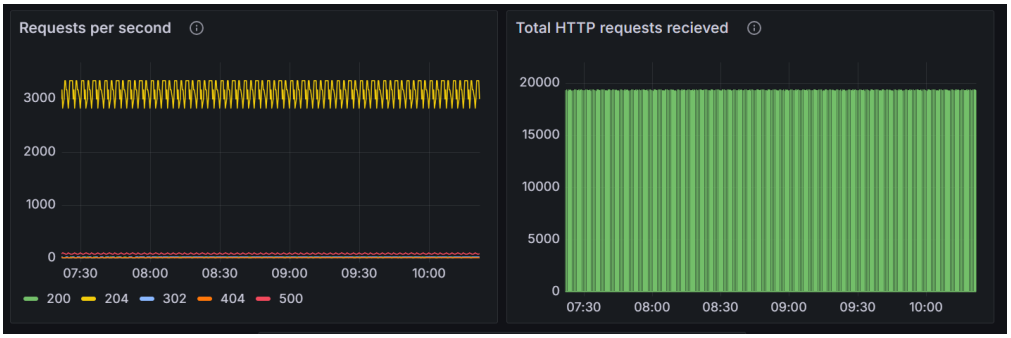


Figure 11: Minitwit Infrastructure Health dashboard - Requests per second and Total HTTP requests received

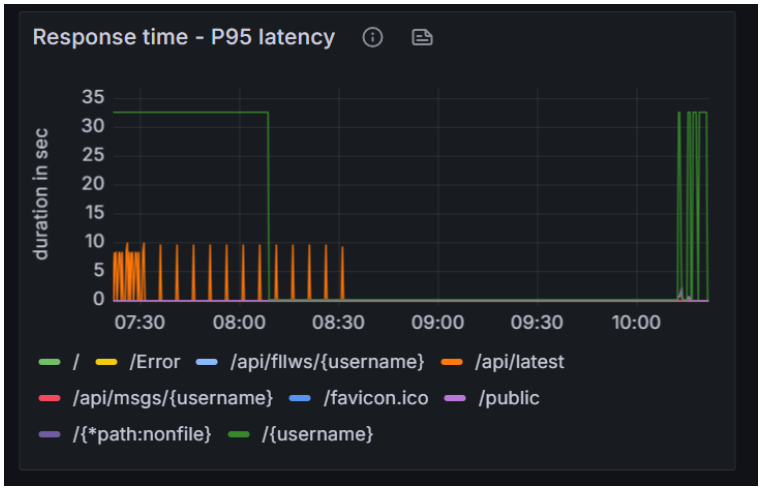


Figure 12: Minitwit Infrastructure Health dashboard - Response time (P95 latency)

2.2.1.2 Minitwit Product Performance Dashboard

In Figure 13,14,15, are images of the dashboard and descriptions of the information it provides the business stakeholders:

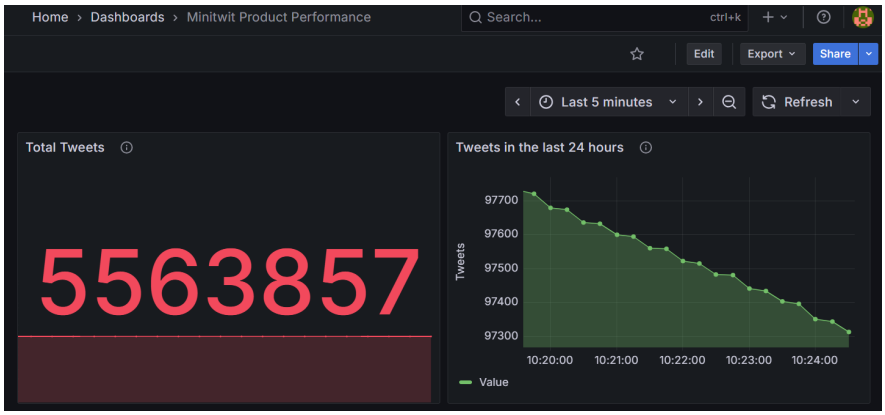


Figure 13: Minitwit Product Performance dashboard - Tweet overview

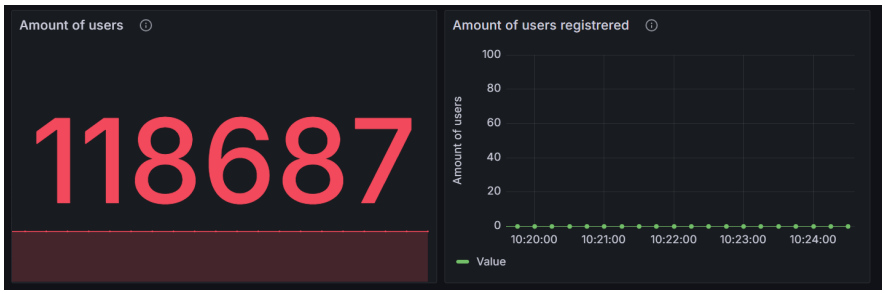


Figure 14: Minitwit Product Performance dashboard - User overview

- KPI Tracking: "Total Tweets" and "amount of users" are direct Key Performance Indicators. In a real application, if registrations drop to zero, the Marketing or Product team needs to know immediately.
- Engagement Trends: The 24-hour tweet increase shows the pulse of the community. A sudden drop might indicate a trend issue or a subtle bug that doesn't trigger an error but prevents posting. The amount of users registerered in the last hour, can be used to analyze if a campaign or trend has had impact or not.
- User Satisfaction: Slow apps lead to lost users, so we include an overview of the latency of HTTP requests to keep the business aligned with the need for technical optimization.

2.3 Logging

Logs from MiniTwit uses a Push-based model, where Serilog directly pushes logs to Elasticsearch. Since Serilog is directly implemented in MiniTwit, this means all MiniTwit replicas are pushing logs to Elasticsearch, where they are collected and displayed in a singular view. Our default logging level is configured to be *Information*, i.e any logs from the MiniTwit application will be logged if it is at the information level or above. We override this for Microsoft, AspNetCore and EFCore where we only log *Warning* and above. MiniTwit logs use the *minitwit-logs* index prefix, making them easy to distinguish from other logs.

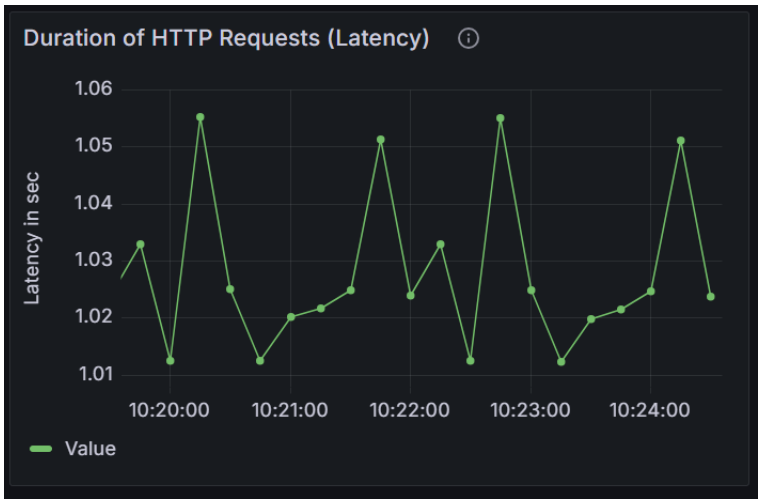


Figure 15: Minitwit Product Performance dashboard - Duration of HTTP Requests (Latency)

2.3.1 What our current logging implementation is missing

Since our infrastructure is running in docker containers, we should also have set up a log-shipper, such as Filebeat, to collect docker container logs and ship them to Elasticsearch.

Furthermore, our actual logging statements in MiniTwit.cs are also quite sparse, with some functions not actually logging anything. Whilst not everything needs to be logged, this should be a more deliberate choice than it is currently.

2.4 Security

Security in our system is enforced on several layers.

We've configured a DigitalOcean firewall for our database, following a default-deny principle, i.e all inbound traffic is blocked, unless manually stated otherwise. Inbound traffic is allowed for SSH on port 22, and droplets with the minitwit-swarm tag on DO is allowed for MySQL on port 3306

Our droplets all have UFW set up, also following a default-deny principle for incoming traffic. During setup of the droplets, the configuration scripts opens specific ports for inbound traffic. The configuration scripts also updates the default ssh settings, limiting retries, reducing the graceperiod and disabling password authentication.

For transport security, we use Let's Encrypt to provision SSL/TLS certificates, ensuring that communication between clients and the application is encrypted over HTTPS. Furthermore we ensure that attempts to access the site through HTTP is redirected to a secure HTTPS connection.

On the application level, we enforce security through frameworks implemented in the codebase. Database access is handled through Entity Framework, which uses parameterized queries to help prevent SQL injection attacks. Razor Pages automatically encodes all HTML, which protects against cross site scripting (XSS) from malicious users. For users of the minitwit application passwords are encrypted using PBKDF2 with salting, which ensures that passwords are securely stored, and safe from brute force attacks. Our code base has .NET analyzers, which picks up on security diagnostics, and our .editorconfig file ensures that any security issues discovered through analysers will be marked as errors, meaning the application will fail to build, if a security issue is present.

Within our CI/CD pipeline we have have two security tools, Trivy for Dockerfiles and images, and CodeQL for static analysis of the codebase. Trivy is run in its own workflow, which scans for misconfigurations in all

Dockerfiles present in the repository. It then builds the images and scans for vulnerabilities in those. If any misconfigurations or vulnerabilities are found, it will fail the workflow. CodeQL runs in its own workflow, which scans the codebase by running a set of queries to find common vulnerabilities in the code base. Both workflows runs when a pull request to main is created, on push to main, and each week on Monday. Running both workflows on a schedule, rather than just on PR and push is relevant, since new vulnerabilities can be discovered any time, and therefore code that was thought to be secure when the code was pushed, may be discovered to be insecure. The weekly workflow helps ensure that we become aware of newly discovered vulnerabilities.

2.5 Availability

3 Reflection Perspective

Describe the biggest issues, how you solved them, and which are major lessons learned with regards to:

- evolution and refactoring
- operation, and
- maintenance

of your ITU-MiniTwit systems. Link back to respective commit messages, issues, tickets, etc. to illustrate these.

Also reflect and describe what was the "DevOps" style of your work. For example, what did you do differently to previous development projects and how did it work?

3.1 Evolution and Refactoring

Remember to write about how changing from one docker container, to using Docker swarm destroyed our old way of monitoring. (OBS! we have written something about monitoring, so just a quick comment on how this destroyed it.)

3.2 Operation

3.3 Maintenance

4 Use of AI

ITU's rules on the use of generative AI apply for this report too. They are described here and in detail here. Please follow them. For your report that means that you have to state which generative AI tools have been used for which task(s) in your projects. Additionally, describe how generative AI tools have been used and briefly reflect and discuss how they supported or hindered your work process.

5 Appendix

5.1 Appendix for CI/CD workflows

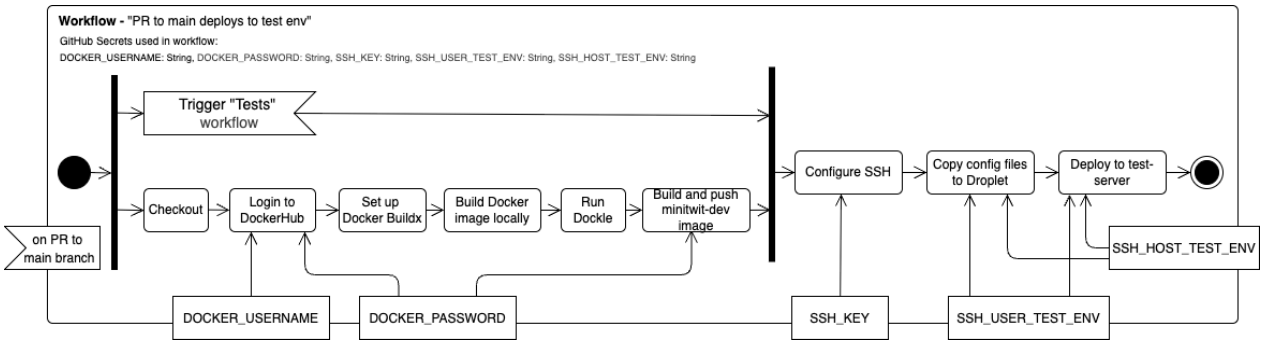


Figure 16: Workflow testing and deploying to test environment

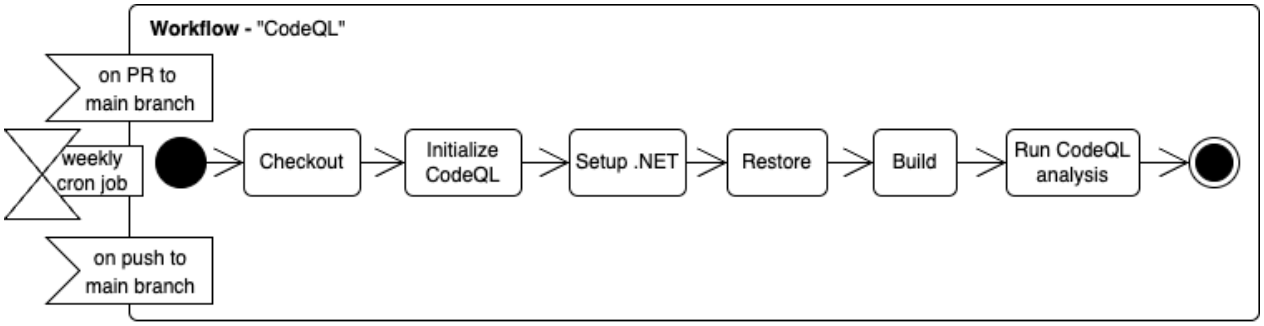


Figure 17: Workflow for CodeQL

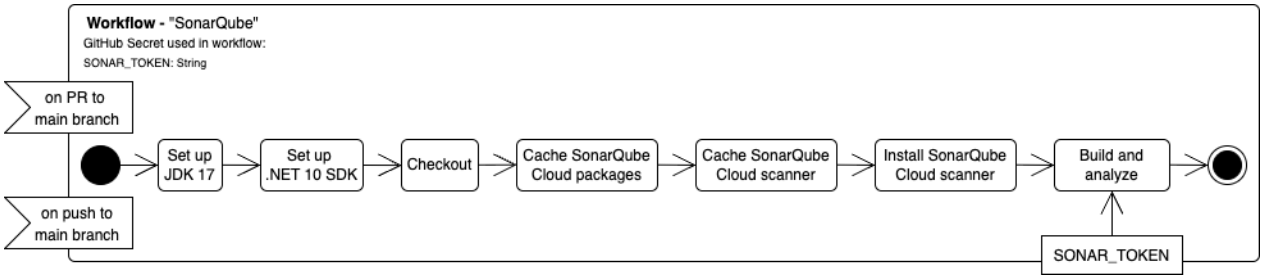


Figure 18: Workflow for SonarQube

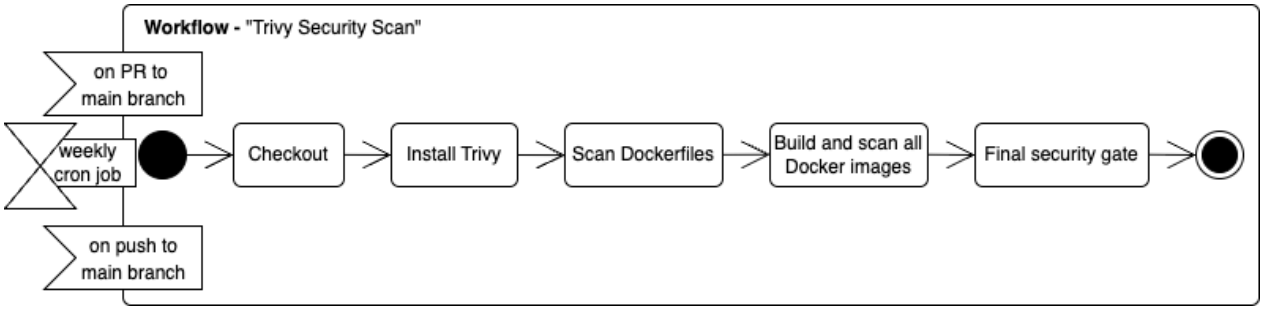


Figure 19: Workflow for Trivy

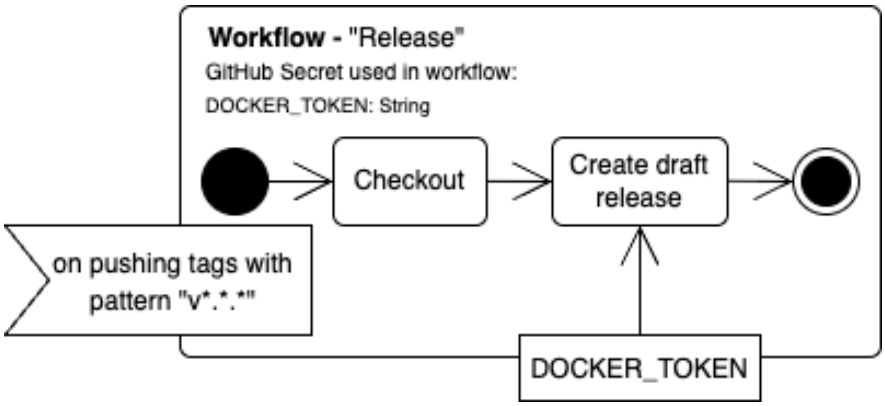


Figure 20: Workflow for drafting release

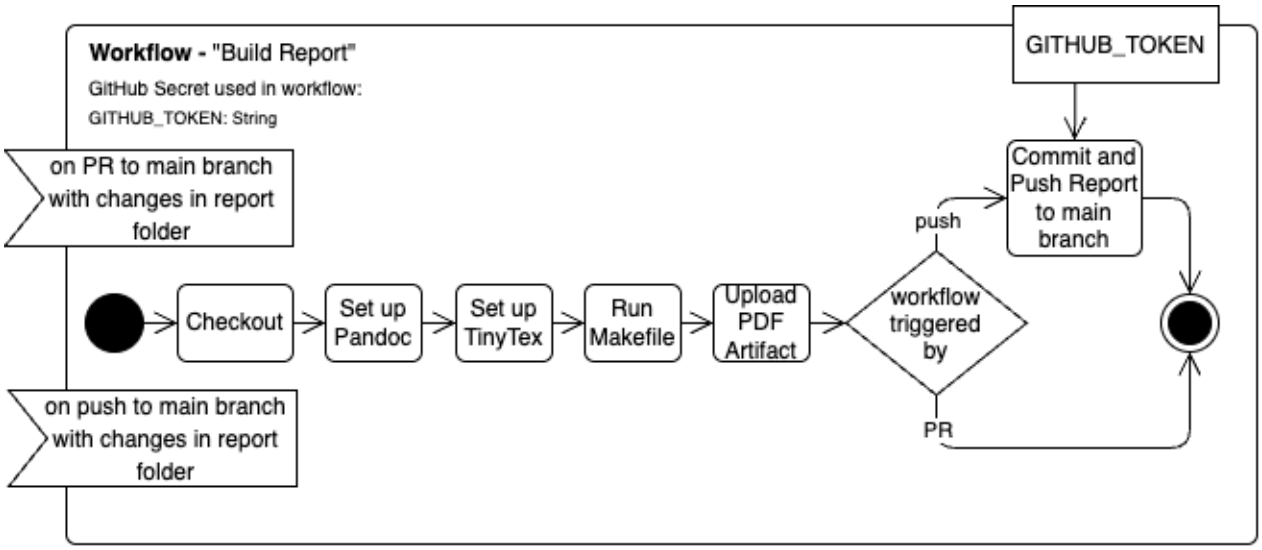


Figure 21: Workflow for generating report